

LAPORAN TUGAS 6

AUTHORIZATION & ROLE BASED ACCESS

CONTROL

Nama: Faisal

NIM: 434241125

Project: API Students - Go Fiber

Repository:

1. Ringkasan Implementasi

Project Modul 5 dikembangkan dengan RBAC berbasis tabel `roles`, `permissions`, dan `role_permissions`. Permission dimuat sekali saat startup ke `PermissionSet`. Keputusan yang tidak membutuhkan data dipasang pada route melalui `RequirePermission`, sedangkan keputusan berdasarkan ownership dilakukan di service.

Entity student memperoleh `owner_id`. Nilainya selalu diambil dari user pada access token ketika POST dan tidak tersedia di request struct, sehingga klien tidak dapat mengaku sebagai pemilik lain melalui mass assignment.

2. Potongan Kode Penting

2.1 Migration permission student

```
INSERT INTO role_permissions (role_name, permission_name) VALUES
('admin', 'student:list'), ('admin', 'student:read:any'),
('admin', 'student:create'), ('admin', 'student:update:any'),
('admin', 'student:delete'), ('staff', 'student:list'),
('staff', 'student:read:any'), ('staff', 'student:create')
ON CONFLICT DO NOTHING;
```

```
ALTER TABLE students
ADD COLUMN IF NOT EXISTS owner_id INTEGER REFERENCES users(id);
```

`owner_id` dibuat nullable agar migration tetap dapat dijalankan saat tabel sudah berisi data lama. Data lama tanpa owner tidak dianggap dimiliki user mana pun, tetapi masih dapat diakses role yang memiliki permission `:any`.

2.2 Middleware permission

```
user, ok := helper.CurrentUser(c)
if !ok { return helper.Fail(c, 401, "belum terautentikasi") }
if !perms.Can(user.Role, permission) {
    return helper.Fail(c, 403, "role ... tidak memiliki hak ...")
}
return c.Next()
```

2.3 Pure ownership rule

```
func CanAccessStudent(current model.AuthUser, ownerID int,
    perms *helper.PermissionSet, anyPermission string) bool {
    if current.UserID == ownerID { return true }
    return perms.Can(current.Role, anyPermission)
}
```

2.4 Service create student

```
current, ok := helper.CurrentUser(c)
// parsing dan validasi request
student, err := s.repo.Create(c.Context(), &req, current.UserID)
```

3. Matriks Hak Akses Students

Endpoint	Admin	Staff	User pemilik	User bukan pemilik
GET /students	200	200	403	403
POST /students	201	201	403	403
GET /students/:id	200	200	200	403
PUT /students/:id	200	200 jika pemilik; 403 jika bukan	200	403
PATCH /students/:id	200	200 jika pemilik; 403 jika bukan	200	403
DELETE /students/:id	204	403	403	403
Semua endpoint tanpa token	401			

Catatan bukti: status di atas adalah hasil yang diwajibkan oleh policy dan perlu dicocokkan dengan output curl setelah migration dijalankan pada database praktikum. Jangan mengganti bagian bukti dengan klaim pengujian sebelum benar-benar menjalankannya.

4. Perintah Pengujian Manual

Setelah login sebagai admin, staff, dan user, simpan access token masing-masing. Contoh:

```
curl -i http://localhost:3000/api/v1/students \
-H "Authorization: Bearer $TOKEN_ADMIN"

curl -i http://localhost:3000/api/v1/students/1 \
-H "Authorization: Bearer $TOKEN_USER"

curl -i -X POST http://localhost:3000/api/v1/students \
-H "Authorization: Bearer $TOKEN_STAFF" \
-H "Content-Type: application/json" \
-d '{"nim":"434241125","name":"Faisal","grade":90,"is_active":true}'

curl -i -X PATCH http://localhost:3000/api/v1/students/1 \
-H "Authorization: Bearer $TOKEN_ADMIN" \
```

```
-H "Content-Type: application/json" \  
-d '{"grade":95}'  
  
curl -i -X DELETE http://localhost:3000/api/v1/students/1 \  
-H "Authorization: Bearer $TOKEN_ADMIN"
```

4.1 Pengujian negatif

```
# Mass assignment owner_id harus diabaikan; owner response tetap user dari token.  
curl -i -X POST http://localhost:3000/api/v1/students \  
-H "Authorization: Bearer $TOKEN_STAFF" \  
-H "Content-Type: application/json" \  
-d '{"nim":"NEG001","name":"Negative Test","grade":80,"is_active":true,"owner_id":1}'  
  
# User bukan pemilik harus memperoleh 403.  
curl -i -X PUT http://localhost:3000/api/v1/students/1 \  
-H "Authorization: Bearer $TOKEN_USER_LAIN" \  
-H "Content-Type: application/json" \  
-d '{"nim":"NEG002","name":"Unauthorized","grade":80,"is_active":true}'
```

5. Analisis

5.1 Mengapa ownership tidak dipindahkan ke middleware?

Pada implementasi ini, `RequirePermission` hanya menerima identitas dari JWT dan nama permission. Untuk mengetahui pemilik student, kode harus memanggil `StudentRepository.FindByID` dan membaca `OwnerID`. Karena repository digunakan oleh `StudentService`, pemeriksaan GET/PUT/PATCH ditempatkan di service agar middleware tidak bergantung pada database maupun struktur entity.

5.2 Risiko pemeriksaan terbagi di route dan service

Risiko konkretnya adalah endpoint baru dapat lupa diberi penjaga, khususnya route ownership yang terlihat tidak memakai `RequirePermission`. Risiko dikurangi dengan menempatkan seluruh mapping endpoint dalam `route/route.go`, memakai pola konsisten, dan menguji pure function authorization secara terpisah. Pengujian matriks endpoint tetap diperlukan untuk memastikan tidak ada jalur yang terlewat.

5.3 Kasus dosen wali dan mahasiswa bimbingan

RBAC saja tidak memadai karena keputusan bergantung pada relasi antara dosen dan mahasiswa tertentu, bukan hanya nama role. Sistem perlu menambahkan atribut/relasi bimbingan di database dan aturan ABAC di service, misalnya mengizinkan akses bila `student.advisor_id == current.UserID`, di samping permission RBAC yang sudah ada.

6. Verifikasi Kode

Perintah	Hasil
go fmt ./...	Lulus

go test ./...	Lulus
go vet ./...	Lulus
go build ./...	Lulus